NetScope 网络嗅探器设计与实现

软件与系统安全 实验 1A 技术报告

独立实现的 macOS 图形化 NPS 与 NPA

C++17 · Qt 6 · libpcap

姓名：于豪杰　学号：202628015059027

2026 年 11 月

摘要

NetScope 是面向 macOS 的独立网络数据包嗅探与协议分析工具。软件直接调用 libpcap 从多个网络接口获取二层帧，不依赖 Wireshark 或 tshark；在用户态完成安全字节读取、分层协议解析、内核 BPF 抓包过滤、布尔显示过滤、TCP 流追踪、IP 分片重组、进程归属、统计分析与异常观察，并提供可选的 Wireshark extcap 互操作桥。Qt 6 界面采用标准三窗格浏览器，将包列表、协议树与十六进制字节进行双向联动。项目满足实验对 GUI、NPS、分层过滤与 OSI 层次理解的要求，并实现了基于 IP 加端口的 TCP 流追踪与基于进程的流归属两项加分功能。

关键词：网络嗅探；libpcap；BPF；协议解析；TCP 流；Qt 6

1 实验目标与项目定位

实验 1A 的核心是实现带图形界面的网络嗅探器，必须具备 NPS 能力，并尽量完善 NPA 能力。NetScope 的定位不是封装或调用 Wireshark，而是从抓包、解析、流状态到交互展示全链路独立实现。这一定位既能展示对 OSI 层次、内核抓包和协议状态的理解，也使每个安全边界和失败模式可被测试。

学生：于豪杰（学号 202628015059027）。项目开源仓库：https://github.com/LockiYu/NetScope。

要求	实现	验收依据
GUI	Qt 6 原生桌面界面，包列表、协议树、Hex 视图与可停靠分析面板	离线 UI 集成测试
NPS	libpcap 实时抓包，支持多网卡、暂停、继续、停止和丢包统计	后端会话与抓包运行器测试
协议过滤	内核 BPF 与应用层逻辑表达式两级过滤	编译验证与过滤器单元测试
TCP 流	五元组归并、握手状态、重传、乱序、RTT、双向重组和时序图	流追踪单元测试与界面测试
进程归属	libproc 采样五元组到进程的映射，包列表与流视图按进程过滤	进程追踪单元测试与界面测试

要求	实现	验收依据
Wireshark 互操作	extcap 接口把 NetScope 暴露为 Wireshark 数据源，pcapng 包注释携带进程	extcap 契约测试、pcapng 单元测试与端到端验证

2 OSI 七层模型与抓包层次

OSI 七层模型把通信拆分为物理、数据链路、网络、传输、会话、表示和应用。嗅探器并不需要在每一层各抓一份：它只在一个确定的层次复制字节，然后在用户态逐层剥离头部。抓包层次决定能看到什么，分析层次决定能解释什么。

层次	抓包位置	本项目的分析产物	边界
物理层	不可见	—	信号、误码、链路速率
数据链路层	BPF 设备复制完整以太网帧	Ethernet II、802 1Q QinQ、ARP、MAC 与 VLAN	驱动丢包与聚合前的状态
网络层	同一帧内的 IP 头部	IPv4 IPv6 地址、选项、扩展头、分片、ICMP	路由表与 NAT 前后差异
传输层	TCP UDP 头部	端口、序列号、标志、窗口、流状态与重传	内核拥塞控制与重传队列
会话/表示/应用	TCP UDP 载荷	HTTP 起始行与头名、TLS 握手元数据、DNS、DHCP	已加密的应用数据

实验讲义用 Linux netfilter 的五个 hook 说明包在协议栈的哪个位置被拦截。抓包点越靠近网卡，字节越原始，能看到分片、重传段与校验和错误；越靠近应用，语义越完整，却已经失去链路层细节。macOS 没有 netfilter，抓包通过 BPF 设备在内核完成同样的角色：BPF 程序先按表达式过滤，再把命中的帧复制到用户态，这正是抓包工具能跟上高速链路的原因。

基于这一认识，NetScope 把抓与析严格分开：抓包层只在链路层取得完整帧并做 BPF 过滤；解析层与流层在用户态重建 VLAN、IP、TCP 与应用的语义。Linux 上更靠近协议栈的抓包方式

（AF_PACKET、eBPF）能获得更多上下文，但也更难移植；在 macOS 上从 BPF 设备抓二层帧，是可见性、权限与可移植性之间的折中。

2 1 思考题：Thor 的能力边界说明了什么

Thor 依靠安装系统描述文件与根证书，把自己变成应用层的 TLS 中间人：被代理的应用信任 Thor 的 CA，于是完成两次握手，客户端与 Thor、Thor 与服务器各一次，HTTP 请求行、头部与正文在 Thor 进程内已经是明文。这就是它能抓取 HTTP HTTPS 应用层包的原因。

但它对某些传输控制协议的内容无能为力，原因不在密码学强度，而在层次与信任边界。其一，QUIC 与 HTTP3 运行在 UDP 443 上，不经过系统 TLS 会话，系统代理与根证书都插不进去；其二，启用证书固定或双向认证的应用会拒绝非预期的证书链，中间人握手直接失败；其三，非 HTTP 的私有二进制协议即使被解密，也需要专门的解析器，而 Thor 没有对应模型；其四，应用层再做一次端到端加密时，中间人得到的只是二次密文。

因此，抓包能力受限于抓包点所在层次以及是否掌握该层密钥，分析能力受限于是否实现了该层语义解析。工具看起来能抓 HTTPS，并不等于能理解 HTTPS。对攻击者，这说明绕过中间人的成本可以很低；对防御方，这说明端到端加密、证书固定与 QUIC 是有效的加固手段；对本实验，这正是 NPS 与 NPA 必须分层实现的原因：NPS 保证字节可见，NPA 保证字节可解释，任何超出抓包层次的断言都是过度承诺。

3 系统架构

项目采用分层架构，核心库不依赖 Qt Widgets，因而协议解析、重组和过滤可在无 GUI 环境下独立测试。界面层只负责交互状态、模型映射和异步调度。

层次	主要组件	责任
采集层	capture_session capture_runner packet_merger	网卡枚举、BPF、多线程抓包、合并、去重与 PCAP I O
解析层	byte_view packet_parser application_parser	边界检查的字节读取与分层协议树生成
分析层	display_filter tcp_flow ip_reassembly	显示过滤、TCP 流状态、分片重组和包对比

层次	主要组件	责任
洞察层	traffic_statistics anomaly_detector reverse_dns process_tracker	聚合统计、可解释观察、异步反向 DNS 与进程归属
界面层	main_window	三窗格浏览、停靠面板、图表、导出和配置持久化

4 数据包采集

4 1 多网卡与抓包生命周期

程序枚举 macOS 可见接口，识别活动、回环、无线和隧道属性。用户可同时选择多个接口，每个接口由独立会话读取，然后按时间戳合并。合并器对短时间窗内的同字节包去重，避免同一包被物理口与虚拟口重复展示。状态机支持 active、paused 和 stopped，停止时主动打断 pcap 读取并回收线程。

4 2 过滤、截断长度与代理边界

抓包截断长度可编辑，默认 262144 字节，即保留完整帧的实用上限。BPF 表达式在开始前编译，无效语法不会进入抓包状态。过滤历史和收藏提供补全。数据面使用 libpcap 直接读取 BPF 设备，不经过 HTTP HTTPS SOCKS 代理；反向 DNS 仅调用系统解析器，遵循 macOS 默认网络路径。

5 协议解析与数据模型

解析器使用不拥有内存的 byte view 和显式大端读取函数。每次读取前都验证偏移与长度，截断包产生警告而不越界。解析结果是递归 protocol node，字段同时保存原始值、显示值、字节范围、校验状态和警告，因此同一模型可支撑树形展示、Hex 反向定位、JSON 导出与异常检测。

层次	已实现协议或元数据	关键处理
L2	Ethernet II 802 1Q QinQ ARP	叠加 VLAN 、MAC 与 ARP 字段
L3	IPv4 IPv6 ICMP ICMPv6	IPv4 选项、IPv6 扩展头、校验和分片
L4	TCP UDP	端口、序列号、标志、选项与校验结果

层次	已实现协议或元数据	关键处理
L7	DNS DHCP HTTP TLS	DNS 名称压缩、DHCP 选项、HTTP 起始行与头名、TLS SNI ALPN

6 过滤与交互分析

显示过滤器与抓包 BPF 分离。它支持 and、or、not、括号、协议存在判断，以及 ip src、ip dst、tcp port、udp port 等字段比较。进程归属也进入过滤语言：process 判断是否存在归属，process.name 与 process.pid 参与比较。应用显示过滤只隐藏行，不删除内存数据，因此清空表达式即可恢复完整会话。

包列表显示绝对本地时间、多网卡来源、归属进程、地址、端口、协议、长度和摘要。列可排序、拖动、隐藏和调宽，布局会自动持久化。反向 DNS 在后台线程查询，界面仍保留原始 IP 供排序与过滤。用户还可标记数据包、添加备注、复制整行或从右键菜单生成过滤条件。

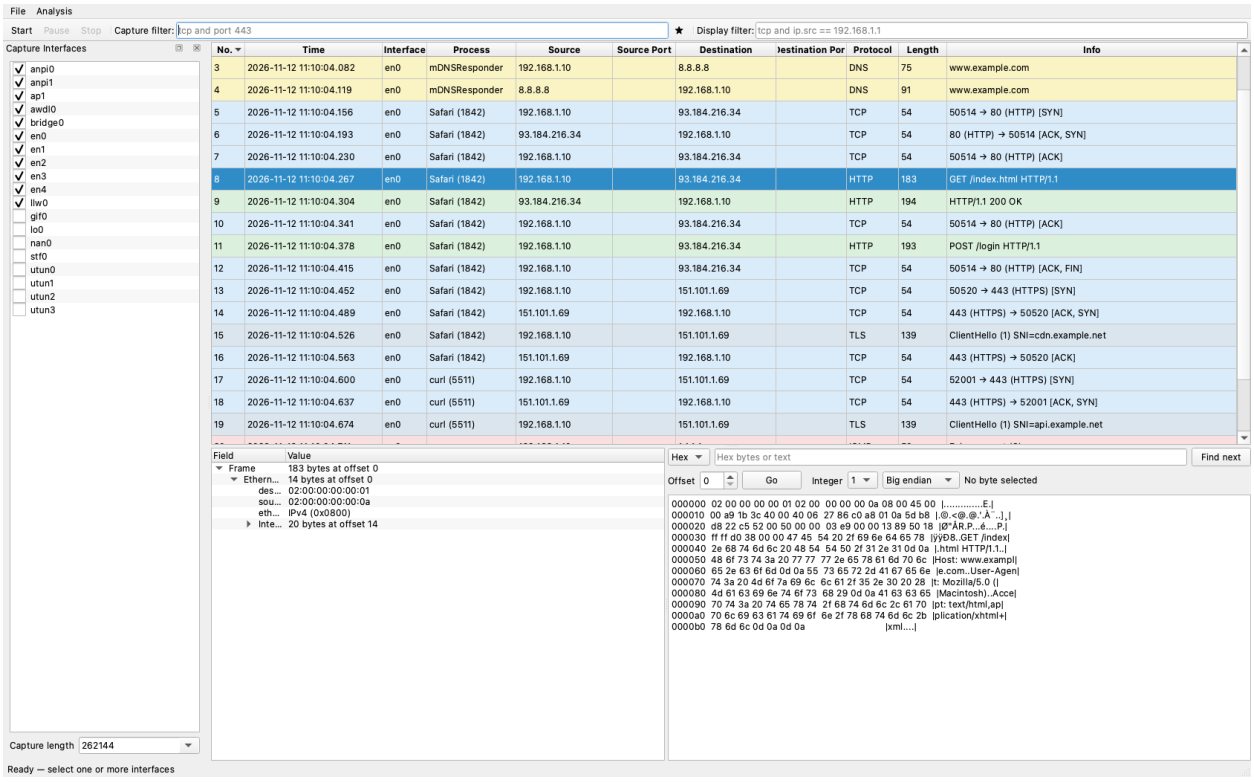


图 1 三窗格主界面：包列表、协议树与十六进制视图双向联动，Process 列显示归属进程

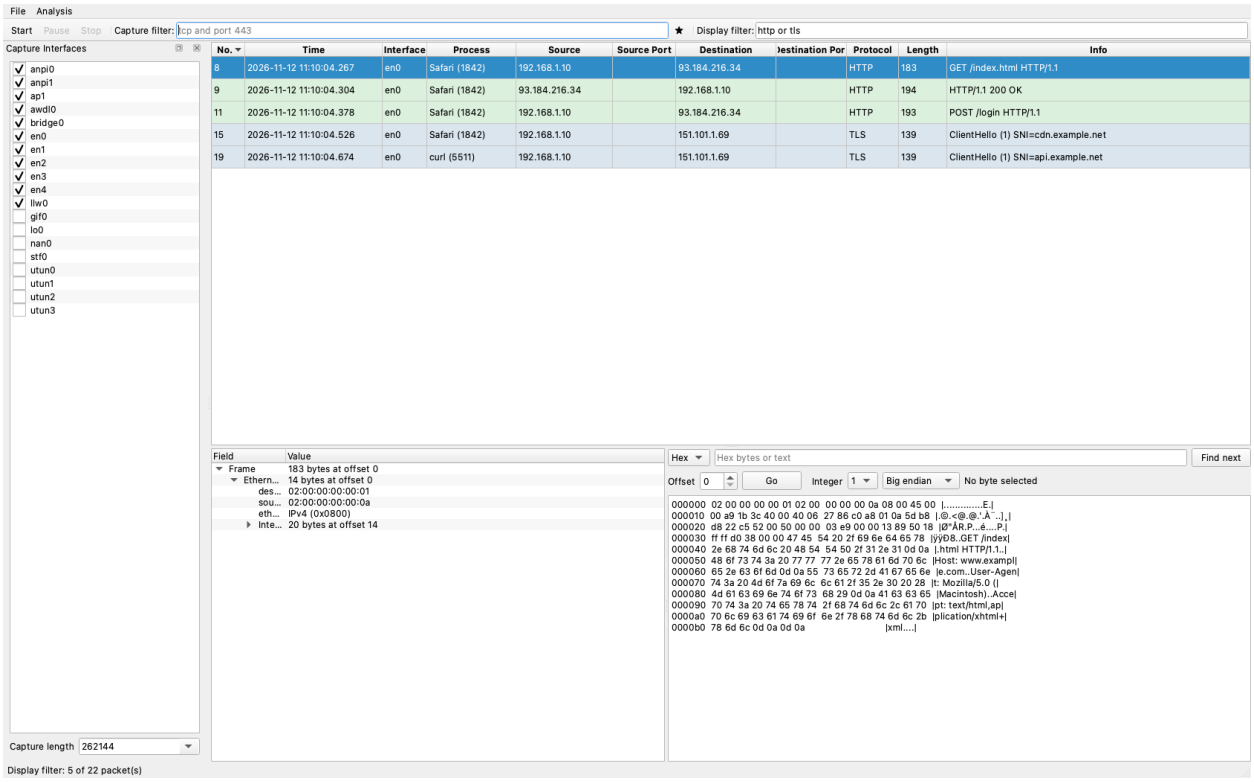


图 2 显示过滤 http or tls：仅隐藏不匹配的行，状态栏给出可见包数与总包数

7 流追踪、重组与时序图

7 1 TCP 流

TCP 跟踪器将正反方向五元组规一化为同一流，根据 SYN、ACK、FIN 和 RST 维护 opening、established、half closed、closed 和 reset 状态。它以 TCP 序列号存储片段，可恢复乱序数据，计数重传和重复 ACK，并从 SYN 与 SYN ACK 估算 RTT。界面提供双向可打印数据、累积字节时序图和单流 PCAP 导出。

7 2 进程归属

实验要求中的流追踪还包括某进程产生的所有 TCP 流。macOS 没有 Linux procfs 那样的统一入口，但 libproc 提供了等价能力：proc_listallpids 枚举进程，proc_pidinfo 配合 PROC_PIDLISTFDS 列出文件描述符，再用 proc_pidfdinfo 与 PROC_PIDFDSOCKETINFO 读取内核中每个 socket 的地址与端口。NetScope 把这些记录整理成传输协议、本地地址与端口、远端地址与端口组成的索引，由后台线程每 5 秒刷新一次。

抓包线程在把帧交给仓库之前，用帧的五元组做双向匹配：出站包用源端点、入站包用目的端点。命中后进程名与 pid 写入包记录，包列表显示 Process 列，流视图同样显示归属进程，并提供进程过滤

框。选择某个进程后，流表只保留该进程的流，配合累积字节时序图即可回答某进程产生的所有 TCP 流。显示过滤语言也扩展了 process 与 proc 两个标识符，支持 process == Safari、process.pid == 1842 这类表达式。

该能力有明确边界：只能读取当前用户有权限的进程，受 SIP 保护的系统进程与其他用户的进程会返回 EPERM；离线 PCAP 不携带进程信息，打开文件时不做事后归属；socket 与进程的对应关系是采样得到的，短连接可能落在两次刷新之间。因此界面把进程归属表述为观察到的归属，而不是审计结论。

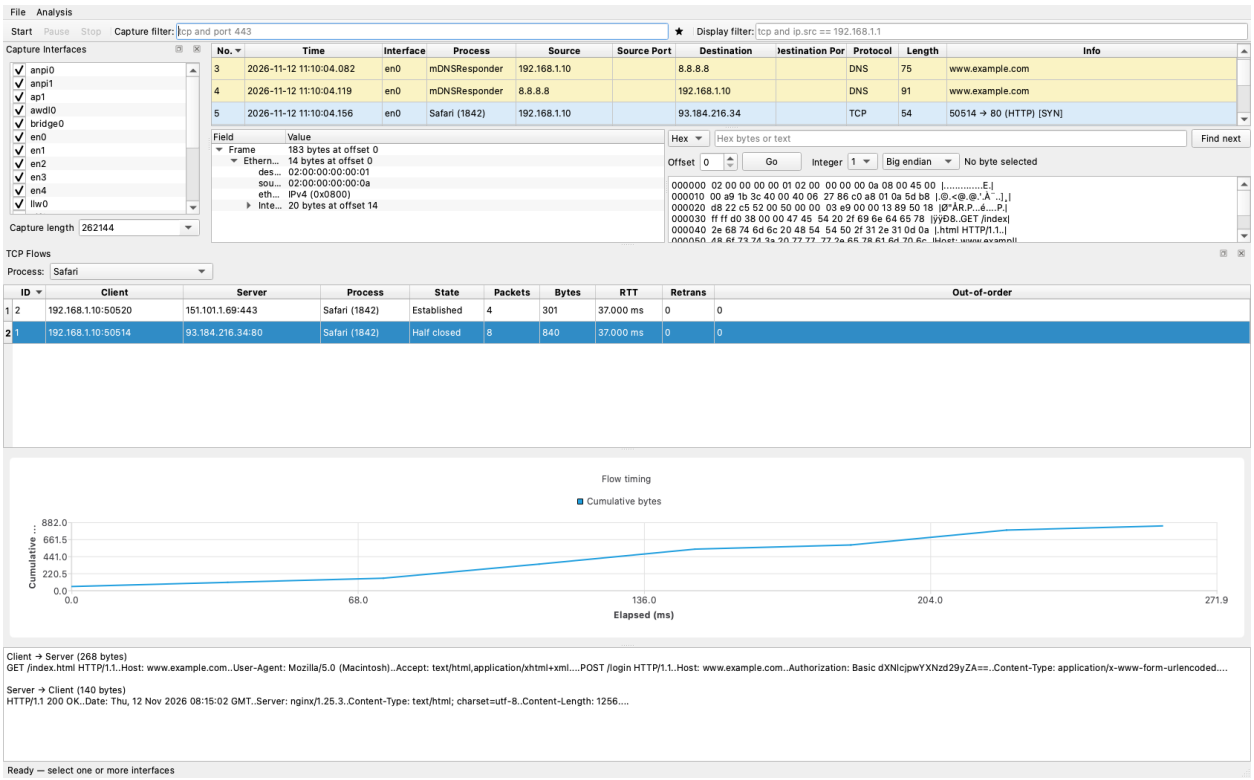


图 3 TCP 流视图：进程过滤为 Safari，选中流后显示重组数据与累积字节时序图

7 3 IP 分片

IPv4 使用源地址、目的地址、Identification 和上层协议建立重组键；IPv6 使用 Fragment 扩展头的 32 位 Identification。重组器接受乱序到达，以字节覆盖图判断缺口，遵循先到字节优先并记录重叠数量。收到尾片且零到终点无缺口时产生完整载荷，并在观察面板记录片数、长度和重叠字节。

8 统计、观察与导出

统计面板实时维护总包数、总字节、包速率、字节速率、协议分布、高频主机、高频端口和长度分桶。观察面板不宣称已证明攻击，而是输出规则名、观察结论、证据和关联包或流。目前包括畸形包、

明文认证头名、端口扫描倾向、高重传比和 IP 分片重组，其端口阈值、时间窗、最小流包数和重传比可调。

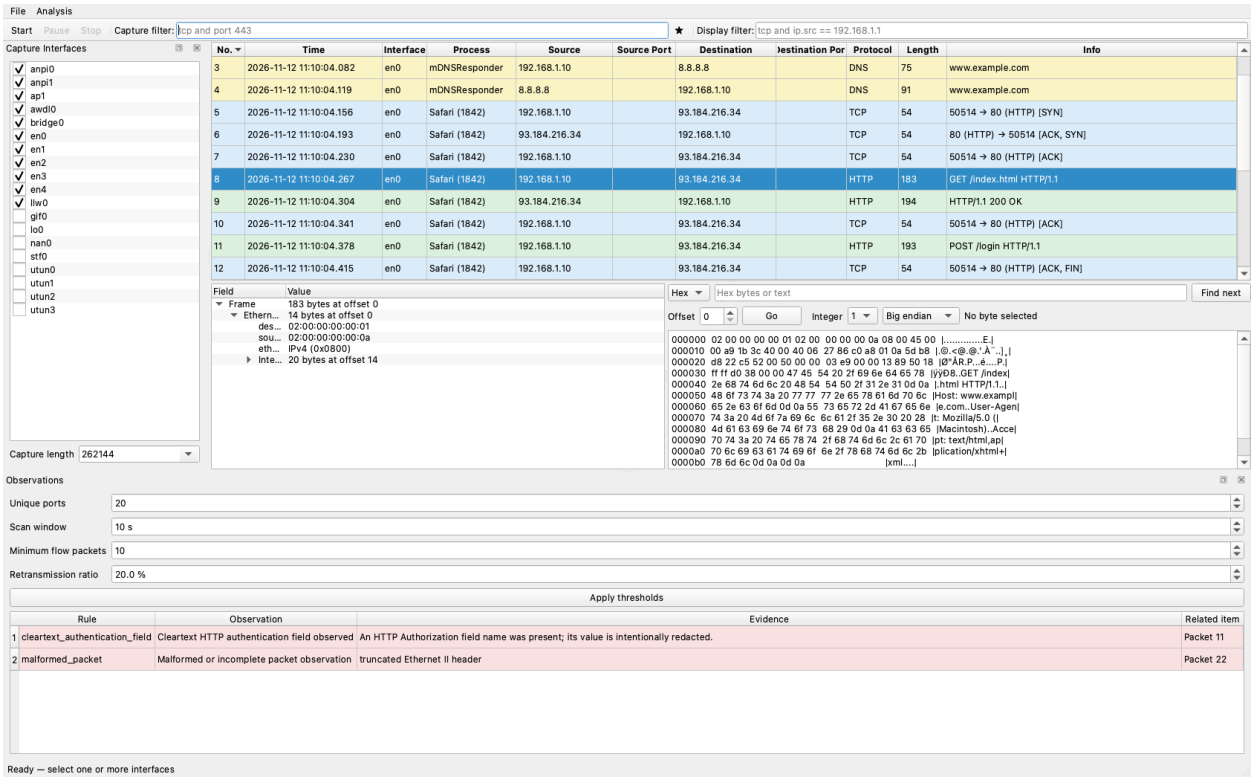


图 4 观察面板：明文认证头名与畸形包两条观察，连同证据、关联包与可调阈值

导出类型	范围	用途
PCAP	全会话	保留原始捕获长度和时间戳
PCAP	当前显示过滤结果	导出调查子集
PCAP	单个 TCP 流	在其他工具中复核流
JSON	选中包的协议树	结构化审计或二次处理
对比视图	两个数据包	按字段路径展示变化与缺失

9 安全性与稳健性

- 所有网络字节读取使用显式边界检查，不对未对齐内存做类型强转。
- 协议解析对截断、无效长度、过多 IPv6 扩展头和异常分片进行有界退出。
- 抓包线程不操作 GUI，数据通过队列切换到 Qt 主线程；反向 DNS 同样异步执行。

- 内存仓库有容量上限，分片待重组表有数量和数据报长度上限。
- HTTP Authorization 只报告字段名存在，不在观察面板回显凭据值。
- 软件不尝试解密 HTTPS，仅显示可见 TLS 握手元数据，避免将元数据分析误表述为内容解密。
- 进程归属仅在内存中与数据包关联，导出的 PCAP 与 JSON 不写入进程信息；读取其他用户进程失败时静默跳过，不提升权限。

10 测试与验证

项目采用测试驱动开发，当前共 108 个自动化测试。单元测试使用手工构造的最小帧覆盖正常、截断、乱序、重传、重叠分片、非法过滤表达式以及进程归属的出站、入站、监听 socket 与未知五元组。集成测试在 Qt offscreen 平台下验证三窗格、抓包控件、反向 DNS、流面板、进程列与进程过滤、统计图、观察、标记、导出与分片完成记录。

测试类别	覆盖内容	预期
构建与解析	工具链、字节读取、协议树、校验和应用层元数据	全部通过
采集与存储	PCAP、网卡、BPF、运行器、合并与仓库	全部通过
分析	显示过滤、DNS、TCP 流、进程归属、统计、观察、IP 重组和包对比	全部通过
GUI	窗口结构、联动、控件、会话、图表与导出	全部通过
动态检查	AddressSanitizer UndefinedBehaviorSanitizer ThreadSanitizer	无报错

11 macOS 构建与运行

开发环境为 macOS，使用 Apple Clang 和 C++17。依赖包括 CMake 3 25 或更高版本、Ninja、Qt 6 Widgets Charts、libpcap 与 GoogleTest。推荐通过 Homebrew 安装依赖，然后执行以下命令。

```
brew install cmake ninja qt libpcap googletest
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Release
```

```
cmake --build build -j
open build/NetScope.app
```

macOS 默认保护 dev bpf 设备。本项目不安装 Wireshark ChmodBPF 或任何开机启动项，而是在演示前使用 sudo chown 和 sudo chmod 临时授予当前用户读写现有 BPF 设备的权限。设备重建或系统重启后该修改会失效，不形成持久权限。若无权限，界面会保留可读错误而不崩溃。

12 Wireshark extcap 互操作

实验说明把基于 Wireshark 的二次开发列为加分项。NetScope 的定位仍是独立实现，因此这一项以互操作桥的形式落地：一个独立的 extcap 可执行文件把 NetScope 的采集能力接入 Wireshark，应用本身不链接 Wireshark，也不改变现有的抓包与解析路径。

extcap 是 Wireshark 定义的进程级接口。Wireshark 通过四个调用与插件交互：列出接口、声明链路类型、查询配置项以及开始抓包。NetScope extcap 复用 list_network_interfaces 枚举网卡，复用 open_live_capture 建立抓包会话，因此 BPF、截断长度、读超时与 immediate mode 的行为与主程序完全一致。

extcap 调用	NetScope 的应答	说明
--extcap-interfaces	extcap 版本行与 interface 行	每张本机网卡暴露为 NetScope: en0
--extcap-dlts	dlt number 等于 1，EN10MB	链路类型为以太网
--extcap-config	filter、snaplen、no-processes、refresh	抓包过滤、截断长度、进程名开关与刷新间隔
--capture --fifo	pcapng 流写入 FIFO	Wireshark 关闭读端时插件自行退出

数据格式选择 pcapng 而不是经典 pcap，是因为 pcapng 的增强包块带有选项区，可以把归属进程写成 packet comment，形如 NetScope process: Safari (pid 1842)。Wireshark 会把它显示在报文注释面板，于是同一个报文既有原始字节，也有采集侧的进程上下文。插件为每个包生成一个增强包块，时间戳使用微秒精度的 64 位时间；进程映射每 5 秒刷新一次，读取其他用户进程失败时按无归属处理，与主程序的边界一致。当前插件只暴露单接口采集，多网卡归并仍由主程序完成。

验证分三层。extcap 契约（接口枚举、链路类型、配置项以及未知接口被拒绝）作为 CTest 用例；pcapng 块编码（块长度自洽、字节序魔数、时间戳、选项与四字节填充）由单元测试覆盖；端到端则

在一次实测中于本机 en0 上通过 FIFO 采集 78 个包，再用独立的 pcapng 实现（scapy）回读，得到 78 个报文与 38 条进程注释，说明第三方解析器可以正确读取该流。

使用方式是运行 tools/extcap/install_extcap.sh，脚本把可执行文件链接到 Wireshark 的个人 extcap 目录（通过 tshark -G folders 探测，回退到 ~/.local/lib/wireshark/extcap）。重启 Wireshark 后，在 Capture Options 中即可看到 NetScope 接口，并可设置抓包过滤、截断长度与进程名开关。

13 演示与验收建议

顺序	操作	展示要点
1	选择物理网卡和 lo0，使用完整包长开始抓包	多网卡、状态切换和绝对时间
2	访问 HTTP 站点并执行 DNS 查询	DNS HTTP TCP UDP 颜色、元数据和反向 DNS
3	输入 BPF 与显示过滤表达式	内核过滤与非破坏式显示过滤的区别
4	选择包树字段和 Hex 字节，搜索文本与解释整数	双向范围联动和安全字节视图
5	查看 TCP 流、重组数据和时序图	五元组流追踪、RTT、重传和导出
6	在流面板按进程过滤，例如只保留 Safari	进程归属与某进程的全部 TCP 流
7	展示统计和观察面板，调整阈值	可解释证据与可配置性
8	标记备注，对比两包，导出过滤 PCAP 与 JSON	调查工作流的完整性
9	在 Wireshark 的 Capture Options 中选择 NetScope 接口并抓包	extcap 互操作与报文注释中的进程归属

14 局限与后续工作

当前版本不解密 TLS 应用数据，也未以插件机制动态加载协议解析器。进程归属只覆盖当前用户可读的进程，受 SIP 保护的系统进程与其他用户进程不可见；归属索引每 5 秒采样一次，短连接可能漏

判；离线 PCAP 不携带进程信息，因此打开文件时不做事后归属。这些限制不影响实验的 NPS、分层过滤、TCP 流和进程流归属要求。后续可增加 QUIC 初始包元数据、长时会话的磁盘流式存储、更精细的 IPv6 分片超时策略，以及基于 macOS Network Extension 的授权进程追踪。

15 结论

NetScope 已完成从 macOS 网络接口到图形化分析结果的完整闭环。它不依赖现有协议分析器，而是以可测试的安全字节解析为基础，将内核 BPF、多网卡采集、协议树、流重组、进程归属、时序图、统计观察和调查导出整合为一个原生桌面工具，并提供 Wireshark extcap 互操作桥覆盖二次开发加分项。实验要求的 OSI 层次理解、分层协议过滤、基于 IP 加端口的 TCP 流以及某进程产生的所有 TCP 流均已落地，同时保留了清晰的安全边界和可扩展架构。